Assignment #5 - Strategy Pattern

Course: ShP-2216 – Software Design Patterns (OP 6B06102 – Software Engineering)
Academic year: 2026–2027 (2nd year, 4th trimester)
Institution: Astana IT University - School of Software Engineering
Type: Individual assignment
Weight: 15% of the 2nd Attestation grade
Language: Java (JDK 17, IntelliJ IDEA recommended)
Submission deadline: 26.10.2026, 11:59 PM
Defense deadline: Last practice class of Week 8

---

1. Learning Objectives

- Explain the purpose of the Strategy pattern, how it eliminates conditional logic, and how it differs from the State pattern (CLO 1).
- Implement interchangeable algorithms behind a common interface (CLO 2).
- Defend your design, including how new strategies can be added without modifying existing code (CLO 3).

2. Task Description

Implement the Strategy behavioral design pattern in Java. Choose one of the options below.

Option A — Suggested topic: Navigation / Route Planning

- A Navigator/Context that plans a route using interchangeable strategies (e.g., CarStrategy, WalkStrategy, CycleStrategy).

Option B — Calculator

- A Calculator/Context that performs an operation using interchangeable strategies (e.g., AddStrategy, SubtractStrategy, MultiplyStrategy), replacing a chain of conditionals.

Option C — Free topic

- Any domain with a family of interchangeable algorithms selected at runtime (e.g., sorting strategies, pricing/discount strategies, compression strategies). Confirm with the instructor first.

3. Technical Requirements

Component	Requirement
Strategy Interface	Declares the algorithm's execution method, common to all strategies.
Concrete Strategies	At least three implementations, each encapsulating one algorithm variant.
Context	Holds a reference to a Strategy and delegates execution to it; must support switching strategies at runtime.
Client	Selects and configures the appropriate strategy — no if/switch on strategy type should remain inside the Context.

4. Clean Code Requirements

Identify and justify at least five Clean Code principles applied in your implementation (see Clean Code, Ch. 10, and the assigned reading on Moodle), for example:

- Elimination of conditional/branching logic in favor of polymorphism
- Meaningful, intention-revealing names for each strategy
- Single Responsibility: each strategy does exactly one algorithm, nothing else
- Context remains algorithm-agnostic (no knowledge of concrete strategy internals)
- Easy extensibility demonstrated (adding a new strategy without touching existing classes)

Include a short before/after or annotated code excerpt for each principle.

5. Deliverables

A. GitHub Repository

- Complete, compiling code
- Clear folder structure and a short README.md (how to build and run the project, how to add a new strategy)
- Incremental commit history

B. Report — short Markdown reflection (Moodle LMS)

Submit the report as a Markdown (.md) file on Moodle LMS by the submission deadline (see Section 7). It must follow this structure:

- 1. Introduction — chosen domain and the algorithm family
- 2. UML class diagram (Context, Strategy interface, Concrete Strategies, Client)
- 3. Clean Code section (Section 4)
- 4. Strategy vs. State — a short paragraph explaining why your problem calls for Strategy rather than State (see Lecture 7)
- 5. Conclusion — brief reflection on flexibility gained vs. classes added
- 6. Link to your GitHub repository

C. In-Class Defense

- Walkthrough of your design
- Live demonstration of runtime strategy switching
- Answering instructor questions, including how Strategy differs from State

6. Grading (100% of this assignment → counted as 15% of the 2nd Attestation)

Component	Weight	What is assessed
Report	10%	Clarity, structure, correct UML diagram
Working & Clean Code	40%	Correct, conditional-free strategy selection; Clean Code adherence
Defense	50%	Understanding demonstrated, quality of explanation, ability to answer questions

Graded on the standard course rubric (Excellent 90–100 / Good 70–89 / Satisfactory 50–69 / Fail 0–49).

7. Submission & Deadline

- **Submission deadline:** 26.10.2026, 11:59 PM. Upload your GitHub repository link and Markdown report to Moodle by this time.
- **Defense deadline:** Last practice class of Week 8

- **Defense is mandatory:** an assignment that is not defended receives 0, and failing to defend assignments may result in not being admitted to the Endterm MCQ (Syllabus, Assignment Defenses Policy).
- **Late submissions:** per course policy, any submission made after the Sunday 11:59 PM deadline receives 0, unless a prior extension was approved for a documented exceptional circumstance. Re-submission by individual arrangement is not permitted.

Note: Week numbers correspond to the official Moodle-published course calendar — confirm the exact calendar dates there.

8. Academic Integrity

This is an individual assignment. Discussing concepts with peers is encouraged, but all submitted code and report content must be your own. Plagiarism or code copying will be handled per university policy and may result in a zero grade and disciplinary referral.

Generative AI tools (e.g., ChatGPT, Copilot, other LLMs) may be used only as a reference for examples and clarification (syntax patterns, conceptual explanations) and must not replace independent work, in line with the AITU Student Honor Code. Any external source used must be properly cited in the report.

9. Resources

- Freeman, E., & Robson, E. (2021). Head First Design Patterns (2nd ed.), Ch. 1 (Strategy)
- Martin, R. C. (2008). Clean Code, Ch. 10
- Lecture 7 slides (Moodle LMS) — Strategy pattern components and comparison with the State pattern
- refactoring.guru/design-patterns/strategy